

TP Slurm 1/2

Nom et prénom :

Veuillez recopier les commandes exécutées et les résultats de ces commandes (et/ou inclure des captures d'écran), EXPLIQUER LA COMMANDE et/ou LE RESULTAT, et répondre aux questions de ce document. Les réponses peuvent être rendues dans le format de votre choix (Word, PDF séparé, papier puis scanné), tant que les numérotations des questions/réponses sont respectées.

*Le document contenant les réponses est à envoyer à **lise.jolicoeur@cea.fr** en fin de TP **OU**, si non terminé, jusqu'au **16/02/2026** (inclus).*

Merci d'indiquer votre NOM dans le nom de fichier du document envoyé.

0. Préparation et validation de l'environnement

Dans cette introduction, nous allons démarrer 4 machines virtuelles :

- **vm0** comme leader et worker slurm
- **vm[1-3]** comme workers slurm seulement

1. Démarrer **4** machines virtuelles avec PCOCC avec le *template* du TP Slurm 1.

```
pcocc alloc -c2 mycentos74-tp-slurm1:4
```

2. Suivez l'avancement du démarrage de la première VM et, si vous êtes curieux, profitez-en pour regarder ce que fait une machine au démarrage :

```
pcocc console
```

3. Une fois que vous voyez un prompt pour votre login, sortez de la commande en faisant Ctrl+C trois fois de suite.

4. Se connecter à la première machine virtuelle **vm0** et vérifier l'état du cluster avec la commande **sinfo -l**.

```
pcocc ssh vm0
```

5. Décrivez le cluster d'après la sortie de la commande `sinfo -l`
6. Vérifier le bon fonctionnement de **slurm** en réalisant une première exécution interactive.

```
srun -n 4 -N 4 hostname
```

si message d'erreur `srun: error: Unable to allocate resources: Invalid account ...` ajouter votre login en tant que user avec la commande qui suit :

```
sudo sacctmgr -i add user name=prenom.nom cluster=ensiie  
account=guests fairshare=10 qos=normal,debug
```

1. Premier script batch

Dans un premier temps nous allons réaliser un premier script shell à utiliser dans un job batch.

1. Créer un premier script affichant le nom du nœud exécutant le script (commande 'hostname') et se terminant avec un code retour 0. Pour commencer, l'exécuter directement dans la session en cours, sans Slurm.

```
vim script-1.sh  
chmod +x script-1.sh
```

2. L'exécuter à l'aide de la commande **sbatch** puis analyser le contenu du répertoire courant et l'état des statistiques d'utilisation de **slurm** (commande 'sacct' par exemple).
3. Modifier le script puis l'exécuter afin d'obtenir un état « FAILED » plutôt que l'état « COMPLETED ».
4. Le modifier à nouveau pour retrouver un code d'erreur « COMPLETED » puis demander l'allocation de **4 tâches** et expliquer/montrer la différence au niveau de l'accounting.

2. Intégration de « step » parallèles

L'exécution d'un script batch demandant l'allocation de plusieurs cœurs nécessite le lancement d'un « jobstep » afin de pouvoir les utiliser en parallèle. Le lancement d'un step se fait par l'intermédiaire d'une commande **srun** intégrée dans le script batch.

Vérifier que plus rien ne tourne entre chaque partie.

1. Dupliquer le premier script batch (exemple de nom : 'script-2.1.sh') et le modifier pour intégrer l'exécution d'un step affichant le nom des machines. L'exécuter avec la même option que dans la question précédente. Afficher le contenu du « listing » obtenu et les données d'accounting. **Commenter les résultats pour nodelist et allocpus.**

```
sacct -j %jobid% -o jobid,jobname,nodelist,allocpus,state
```

2. Dupliquer le script précédent (exemple de nom : 'script-2.2.sh') et ajouter un second step exécutant une attente de 120 secondes (sleep 120). Constaté et expliquer le déroulement de l'exécution à l'aide des commandes squeue et sacct puis afficher et analyser les données d'accounting une fois le job batch terminé.

Utiliser les options '-s' et '-i 30' de squeue pour voir l'évolution du job/jobstep

```
sacct -j %jobid% -o  
jobid,jobname,nodelist,allocpus,state,start,end,elapsed
```

3. Dupliquer le script précédent (exemple de nom : 'script-2.3.sh') et remplacer les deux steps par une boucle de 3 steps qui font un sleep de 120 secondes. Le lancer et constater le fonctionnement. Utiliser la commande **scancel** pour arrêter prématurément les 3 steps. Expliquer la sortie de sacct.

Utiliser l'option (-o state%30) de sacct pour imprimer à l'écran l'état en entier

3. Utilisation des QOS

L'objectif est ici de comprendre le comportement des politiques d'ordonnancement en utilisant différentes QOS pour favoriser le démarrage de certains jobs. Le cluster Slurm du TP est configuré de manière à autoriser 2 QOS, une par défaut et une optionnelle. Vérifier que plus rien ne tourne entre chaque partie.

1. Vérifier l'état du service **slurmd** nécessaire au fonctionnement de sacctmgr. Décrire les différences entre les deux QOS.
La/Lesquell/es pouvez-vous utiliser ? Quelle est votre QOS par défaut ? Vérifier que c'est bien le QOS par défaut.

Le script dans /home/shared/lise.jolicoeur/parse_sacctmgr.sh permet de visualiser de manière plus lisible la sortie de la commande 'sacctmgr'. N'hésitez pas à le copier dans votre VM (pcocc scp) et à l'utiliser/le modifier (ex : ./parse_sacctmgr.sh show qos)

```
sacctmgr show qos
```

```
sacctmgr show assoc format=cluster,account,user%30,qos,defaultqos  
srun -n 1 hostname  
sacct -j %jobid% -o jobid,user,jobname,qos
```

2. En vous basant sur la documentation, lancer deux fois de suite le script 'script-2.2.sh' en demandant une allocation de ressources 'exclusive' de **4 tâches** sur **4 nœuds**, puis une troisième fois en demandant l'utilisation de la QOS 'debug'. Constater les priorités et expliquer l'ordre d'exécution des 3 lancements à l'aide des commandes 'sprio' et 'squeue'.

```
squeue -O jobid,state,qos,timeused  
sprio
```

3. Lancer maintenant deux fois de suite le même script **sans** allocation exclusive de ressources et **sans** utiliser la QOS debug. Que constatez-vous ?

```
squeue -O jobid,qos,timeused,state,reason
```

4. Faites la même chose en utilisant la QOS debug. **Que constatez-vous ? Donner une explication en se basant sur les propriétés des QOS.**

```
squeue -O jobid,qos,timeused,state,reason
```

4. Comportements aux limites

L'idée est maintenant de constater le comportement de Slurm lorsqu'un job batch ne se comporte pas de la façon attendue ou que le système lui-même introduit des problèmes. Vérifier que plus rien ne tourne entre chaque partie.

1. Lancer le script 'script-2.3.sh' en demandant 4 tâches et un temps d'allocation de ressources ('total run time') de 1 minute seulement (man sbatch pour trouver l'option). Constater le comportement à l'aide des commandes squeue et sacct jusqu'à sa terminaison. Quel est le « state » associé au job une fois terminé ? Combien de steps ont été exécutées ?
2. Lancer un script avec une seule tâche (-n) consommant 128Mo de mémoire dans un step en spécifiant une demande de 100Mo de mémoire PAR CPU maximum (man sbatch pour trouver l'option). Constater et expliquer les états de sortie ainsi que le contenu du listing obtenu.

```
% cat gen100M.sh  
#!/bin/bash
```

```
srun dd if=/dev/zero of=/dev/shm/128Mo bs=1M count=128  
sleep 30  
rm /dev/shm/128Mo  
exit 0
```

3. Lancer le script script-2.3.sh en demandant 4 tâches sans restriction de temps d'exécution ni de mémoire puis provoquer le reboot de la vm1.

```
sudo ssh vm1 reboot
```

Constater le comportement à l'aide des commandes squeue et sacct jusqu'à sa terminaison. Quel est le « state » associé au job une fois terminé ?

5. Gestion des nœuds

L'état actuel du cluster n'est pas complètement opérationnel. Le reboot de vm1 a changé son état et ne le rend plus éligible à l'allocation. Il est donc nécessaire de corriger ce problème pour pouvoir à nouveau l'utiliser. Vérifier que plus rien ne tourne entre chaque partie.

1. Lancer un script nécessitant les 4 nœuds exclusivement et constater l'état du job.
2. Remettre le nœud vm1 en « production » à l'aide de la commande scontrol et commenter les conséquences sur l'état du job précédemment lancé.

```
sudo scontrol update nodename=vm1 state=idle  
  
scontrol show jobid=%jobid (au cours du job)
```

3. Réaliser la même opération de 4.3 mais en ajoutant le paramètre « --requeue » (voir « man sbatch ») à sbatch au moment de la soumission. Remettre ensuite le nœud en état de « production ». Que constatez-vous pour le job? Quelle est la différence avec 4.3 sans « --requeue » ?

Décrivez les commandes utilisées afin d'observer l'état du cluster et leurs sorties.

4. Lancer deux fois de suite le script script-2.2.sh en demandant une allocation de ressources « exclusive » de 4 tâches sur 4 nœuds, Puis « drainer » un nœud à l'aide de la commande scontrol. Décrire l'état des nœuds à l'aide de la commande sinfo avant et après la fin du premier job et l'état du second job une fois le premier terminé. Remettre ensuite le nœud « drained » en production **(regarder dans « man scontrol » pour trouver les syntaxes exactes).**

Décrivez les commandes utilisées afin d'observer l'état du cluster et leurs

sorties.

5. Lancer le script `script-2.2.sh` en demandant une allocation de ressources de 4 tâches sur 4 nœuds puis passer un des nœuds à l'état « down ». Donner l'état des nœuds. Que constatez-vous sur l'état du job ?

Décrivez les commandes utilisées afin d'observer l'état du cluster et leurs sorties .

6. En déduire d'après 4 et 5 la différence entre mettre « down » un nœud et « drainer » un nœud. Remettre ensuite le nœud en état de « production ».

6. Test du Backfilling

Le cluster est pleinement opérationnel sans aucun job en cours d'exécution. Il est maintenant possible de monter un scénario permettant d'illustrer le fonctionnement du « backfilling ». Vérifier que plus rien ne tourne entre chaque partie. Pour cela, il suffit de créer:

- 1 job « `bkf-1.sh` » nécessitant une partie des ressources pour une période de temps relativement longue.
- 1 job « `bkf-2.sh` » nécessitant l'intégralité des ressources du cluster et de grande priorité.
- 1 job « `bkf-3.sh` » moins prioritaire que le second nécessitant l'espace non occupé (ou moins) du premier job pour une plage de temps suffisamment longue pour ne pas être « backfilled ».
- 1 job « `bkf-4.sh` » moins prioritaire que le second nécessitant l'espace non occupé (ou moins) du premier job pour une plage de temps suffisamment courte pour être « backfilled ».

Aide : Les seules options `sbatch` à utiliser dans cet exercice sont celles vues dans les parties précédentes.

1. Mettre en place le scénario illustrant le principe du « backfilling » :

Lancer les 4 jobs dans l'ordre qui vous semble approprié pour créer un scénario de « backfilling ». Afficher les priorités des jobs en attente avec `sprio` pour vérifier la plus grande priorité du 2ième job.

Fournissez l'ordre de passage des 4 jobs pour montrer que le 4ième job a été « backfillé » (passé avant un job plus prioritaire). Décrivez les commandes utilisées afin d'observer l'état du cluster. N'hésitez pas à inclure la sortie des commandes effectuées et/ou des impressions d'écran montrant la suite des événements.

2. Réaliser à nouveau le scénario en annulant l'exécution du premier job une fois la mise en exécution d'un nouveau job. Qu'en déduisez-vous sur les limitations du « backfilling » ?
3. Réaliser à nouveau le scénario sans spécifier les temps d'allocation souhaités. Qu'en déduisez-vous sur les besoins du « backfilling » ?